24

关于内存虚拟化的总结对话

学生：（咽了一口唾沫）哇，内容可真多。

教授：是啊，然后呢？

学生：呃，这么多东西，我怎么记得住啊？我是说，为了应付考试？

教授：天哪，但愿你不是为了这个才去记的。

学生：那我为什么要记呢？

教授：拜托，我还以为你不会这么想呢。你来这儿是为了学点东西，这样等你将来走出校门，才能明白系统实际上是怎么工作的。

学生：嗯……您能举个例子吗？

教授：当然！我读研究生的时候，有一次和几个朋友测内存访问要花多长时间，结果时不时会测出比预想高得多的数字。要知道，我们原以为数据全都老老实实地待在二级硬件缓存里，访问起来应该非常快才对。

学生：（点头）

教授：我们怎么也想不通是怎么回事。遇到这种情况该怎么办？很简单，去问教授！于是我们去找了我们的一位教授，他看了看我们画出来的图，只说了一句："*TLB*。"啊哈！没错，是 *TLB* 未命中！我们怎么就没想到呢？只要对虚拟内存的工作原理有一个好的模型，各种有意思的性能问题都能诊断出来。

学生：我想我明白了。我正努力在脑子里建立这些心智模型，弄清楚各种东西是怎么工作的；这样将来我自己出去干活时，就算某个系统的表现和预期不太一样，我也不会吃惊。我甚至应该光靠想一想，就能预料到系统会怎样运行。

教授：没错。那你学到了什么？关于虚拟内存怎么工作，你的心智模型是什么样的？

学生：嗯，我想我现在已经很清楚进程引用内存时会发生什么了。您说过很多次，每次取指令都会引用内存，显式的加载和存储也一样。

教授：听起来不错，接着说。

学生：嗯，有一点我会永远记住：我们在用户程序里看到的地址，比如用 *C* 写的程序……

教授：难道还有别的语言吗？

学生：（接着说）……是是，我知道您喜欢 *C*，我也喜欢！总之，我刚才是说，我现在真正明白了：我们在程序里能看到的所有地址都是虚拟地址；我作为程序员，看到的数据和代码在内存中的位置只是一种假象。我以前觉得能打印出指针的地址很酷，现在却觉得挺泄气：那只是个虚拟地址！数据真正所在的物理地址，我根本看不到。

教授：是看不到，操作系统肯定会把它藏起来，不让你看见。还有呢？

学生：嗯，我觉得 *TLB* 是非常关键的一环，它给系统提供了一个存放地址转换的小型硬件缓存。页表通常相当大，所以只能放在又大又慢的内存里。要是没有 *TLB*，程序肯定会慢得多。看来真正让内存虚拟化变得可行的是 *TLB*。我没法想象怎么构建一个没有 *TLB* 的系统！而且一想到有的程序工作集超出了 *TLB* 的覆盖范围，我就不寒而栗：那么多 *TLB* 未命中，简直不忍直视。

教授：是啊，快捂住孩子们的眼睛！除了 *TLB*，你还学到了什么？

学生：我现在还明白了，页表是必须了解的数据结构之一；不过它说到底也只是个数据结构，这意味着几乎什么结构都可以拿来用。我们从简单的结构讲起，比如数组（也就是线性页表），一路讲到多级页表（看起来像树），甚至还有更疯狂的，比如放在内核虚拟内存里、本身也能换页的页表。这一切都只是为了在内存里省下一点空间！

教授：确实如此。

学生：还有一件很重要的事：我认识到，地址转换用的结构必须足够灵活，程序员想怎么用自己的地址空间，它都得能支持。从这个意义上说，多级页表这样的结构就非常理想：只有当用户要用到地址空间的某一部分时，它才会为这部分创建页表空间，所以几乎没有浪费。更早的尝试，比如简单的基址和界限寄存器，就是不够灵活；这些结构必须符合用户对虚拟内存系统的期望和需求。

教授：这个角度很好。那我们学的那些换出到磁盘的内容呢？

学生：嗯，这部分学起来当然很有意思，了解页面置换是怎么工作的也有好处。有些基本策略算是比较显而易见的（比如 *LRU*），但构建一个真正的虚拟内存系统好像更有意思，就像我们在 *VMS* 案例研究里看到的那样。不过不知怎么的，我觉得机制更有意思，策略就没那么有意思了。

教授：哦，为什么呢？

学生：嗯，就像您说的，归根结底，策略问题最好的解决办法很简单：买更多内存。但机制你必须弄懂，才能知道东西到底是怎么运作的。说到这个……

教授：嗯？

学生：呃，我的电脑最近跑得有点慢……而且内存其实也不算贵……

教授：哦，好吧好吧！给你几块钱。去给自己买点 *DRAM* 吧，小气鬼。

学生：谢谢教授！我再也不会换出到磁盘了；就算换，至少也知道背后是怎么回事！